

卒業研究論文要旨（シングルカラム形式の例）

[10 行以上, A4 1~4 ページ シングルカラム・図表があってもよい: 表紙の次のページへ]

(例) 人とコンピュータのインタフェースとして, キーボードやマウスの他に, ペンを用いた入力, 音声による入力, ジェスチャーを用いた入力などが挙げられる. これらのインタフェースは, コンピュータに指示を与えるために手や足, 口などの身体を実際に動かす必要があり, 身体を自由に動かせない人は, 十分に利用できなかった. しかし近年では, 体を動かすことを想像するだけでコンピュータに指示を送ることができる BCI (Brain Computer Interface) 技術が進展し, 身体を自由に動かせない人であっても, 指示を想起するだけで, コンピュータに指示を送ることが可能になりつつある. 従来の研究では, EEG (Electroencephalogram) の電極配置に基づいて変換された行列や画像が入力データとされていたが, この手法が最適な入力である確証がなかった. そこで本研究では, 従来の手法に代わる入力行列を, 相互相関係数を用いて定義し, 運動想像タスク分類システムの実装と評価を行った. 実装には公開されている PhysioNet EEG Dataset を利用した. PhysioNet EEG Dataset は運動を想像させるタスクを行っている間に取得された EEG 信号が記録されている. 従来研究に基づき, PhysioNet EEG Dataset に記録されている EEG 信号を入力として, 5 種類の運動想像タスクを推測するシステムを畳み込みニューラルネットワークとリカレントニューラルネットワークを利用して実装した. 本研究では, 相互相関係数を用いて電極間の相関を算出し, 隣接している電極の相関が大きいほど, 値が大きくなるような指標を定義した. さらに指標に基づいて入力行列を定義し, 本研究で定義した入力行列を使用した場合におけるシステムの精度と, 従来の EEG 電極配列に基づいた入力行列を使用した場合におけるシステムの精度を比較した. その結果, 本研究で定義した入力行列を使用したシステムの精度が 10% 高いことを示した.

卒業研究論文要旨（ダブルカラム形式の例）

[10 行以上, A4 1～2 ページ ダブルカラム・図表があってもよい: 表紙の次のページへ]

1. はじめに

ここから本文を入力する。原則として文字の大きさは、9～12 ポイント、行間は 1 行。

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ

2. 理論、背景（必要に応じて）

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああ

3. 手法（必要に応じて）

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ
ああああ

5. 結果（必要に応じて）

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ

4. 調査、実験（必要に応じて）

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ

6. 考察、検討、結論（必要に応じて）

ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああああああああああああああああああああ
ああああ

参考文献（必要に応じて）

Towards Real-Time Detection of Fileless Malware Using Instruction Analysis

Tomohito ITO (15817009)

Tobe Laboratory

1. Introduction

In recent years, the number of detected fileless malwares, which do not depend on files, has been increasing. Fileless malware is executed in memory of a computer without storing an executable file on disk. Because it does not rely on files, it is difficult to detect it by conventional security software that uses signature-based prevention and detection methods. In order to reduce the damage caused by fileless malware, there is a study that detects the causative script before the fileless attack starts [1]. However, it is not perfect and cannot prevent the attack when PowerShell is launched and the attack starts. Therefore, it is desirable to detect PowerShell attacks in real time when they are launched.

Based on the above background, I have developed an analyzer that can detect fileless malware in real time using memory analysis. In particular, I analyze the instructions executed by the malware and detect instructions accessing the registries.

2. Related Work

Bazzi et al. [1] detected the causative script before the fileless attack is launched. Due to the nature of fileless malware, it uses PDF files and Microsoft Office macros to launch attacks. In their study, it detects fileless attack scripts from PDF files. It supports only PDF files, and cannot handle attacks in Microsoft Office. There is also a related study [2] that detects fileless malware in Linux. In this study, the detection is made possible by monitoring the system calls, but only fileless malware that uses specific system calls can be detected.

In contrast to the both studies, this approach is analyzing the instructions executed by the malware, thereby accommodating the real-time determination of malfunctions

3. Design

The key to real-time detection of malware in this study is based on detecting PowerShell processes and analyzing their memory images to detect access to the registry. The proposed method detects fileless malware from the accessed registry. The proposed method consists of three major operations: process monitoring, memory image acquisition, and memory image analysis. In the process monitoring, the entire process is monitored, and when a PowerShell process is detected, its process ID is retained. In the memory image acquisition, it acquires the memory image of the process from the acquired PowerShell process ID. The acquired memory image is analyzed to detect which registry was accessed. It determines whether the process is a fileless malware by checking whether it accesses a registry that is frequently accessed by fileless malware. The overall stream of processing for detecting fileless malware is shown in Fig. 1.

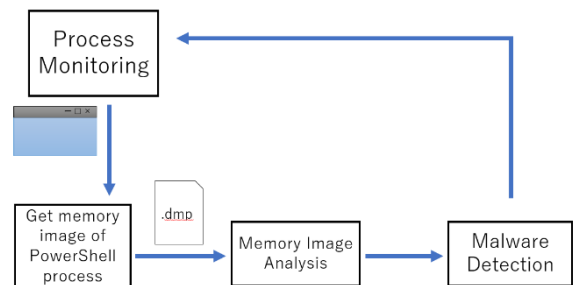


Figure 1 System Overview of Analyzer

4. Preliminary Experiment

I investigated whether the proposed method can detect fileless malware. In this experiment, I prepared in advance the Analyzer, which automates the analysis of PowerShell memory image acquisition, and Poweliks, a fileless malware to test whether it can be detected. In the experiment, I set security software such as Windows

Defender not to run beforehand, and executed Poweliks while Analyzer was running.

As a result of the experiment, the memory image could not be acquired, so it could not be detected. This is probably because the speed at which the Analyzer acquires the memory image is slower than the execution speed of PowerShell.

5. Evaluation Experiment

Based on the results of the experiments described in the previous section, I investigated the execution speed at which the proposed method can be used for detect access to the registry. In this experiment, I created an Analyzer that script to access the registry, Registry::HKLM\Software\Microsoft\Windows\CurrentVersion\Run, which automates the memory image acquisition and analysis of PowerShell, in advance. Subsequently, I ran the scripts with the Analyzer running. The reason why I chose HKLM\Software\Microsoft\Windows\CurrentVersion\Run as the registry to be accessed in the experiment is that filthy malware such as Poweliks will first write an entry in the Run key of the registry. As a result of the Analyzer, when the execution speed of the created script for registry access was slower than 300 ms, the registry that was accessed was present in the handle information. Therefore, when the execution speed of the script was slower than 300 ms, I detected the access to the registry. The graph of multiple access to registry is shown in Fig. 2. As seen in the figure, Analyzer can detect that PowerShell has accessed the registry by analyzing the memory image.

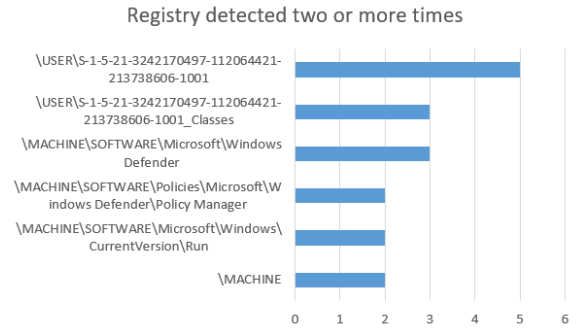


Figure 2 An example of a detected registry

6. Conclusion

In this paper, I described a method for detecting fileless malware in real time by analyzing registry accesses in real time. I verified that I could detect fileless malware when the execution speed of the script was less than 300ms. However, the Analyzer does not support the execution speed of fileless malware. Therefore, it is desirable to improve the speed of the Analyzer. In conclusion, this method can detect FLM if the processing speed of Analyzer is increased. As this future work, I will consider improving the processing speed by creating the Analyzer using a compiler language, and detecting access to the registry by stopping the PowerShell process when it is detected and dumping it while moving.

Reference

- [1] A. Bazzi and Y. Onozato, "IDS for Detecting Malicious Non-Executable Files Using Dynamic Analysis", 15th Asia-Pacific Network Operations and Management Symposium (APNOMS), 2013.
- [2] K. Tanaka and T. Saito, "Mitigation of Fileless-Malware in Linux", Computer Security Symposium 2018, No.2, pp.601-606, 2018.